```
import * as React from "react"
import { createRoot } from "react-dom/client"
import App from "./app"

document.addEventListener("turbo:load", () => {
  const container = document.getElementById("react-element")
  if (container) {
    const root = createRoot(container)
    root.render(
      <App
        rowCount={parseInt(container.dataset.rowCount || "0", 10)}
        seatsPerRow={parseInt(container.dataset.seatsPerRow || "0", 10)}
        concertId={parseInt(container.dataset.concertId || "0", 10)}
      />
    )
  }
})
```

In the first case, the previous version had `if (document.getElementById("react-element"))` and in this version, we do the `getElementById` first and do the `if` statement based on the resulting variable existing. We're changing this so that TypeScript can recognize that now the `if` statement is a type guard, which it can do for the variable, but apparently can't do for the entire statement. TypeScript now realizes that inside the `if` statement, `container` can't be `null`, so the code now passes the `null` check.

In the other cases, we're using the or operator (`||`) and the fact that undefined values are false to specify a value to be passed on if the lookup is undefined, preventing an undefined value from being passed to `parseInt`.

The `SortController` has a similar issue, also solvable with the same solution in the `sortValue` method.

The `debounce` function has a few problems. One is that the first line of the code sets `timeoutId = null`, which, on the one hand, TypeScript now uses inference to assume the type will always be `null`, and on the other hand, the later call to `clearTimeout` wants the default value to be `undefined`. Also, we want to specify the `window.setTimeout` function, which explicitly returns a number for the `timeoutId` rather than the one from Node.js that TypeScript is going to choose by default.